ADVANCED SQL QUERY IMPLEMENTATION

OBJECTIVES

1. Implement subqueries and nested queries.
2. Find products above average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

.

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query. The inner query calculates a result that is used by the outer query. The following output displays products priced above the average price and customers who have placed orders.

```
SQL> SELECT
  2      ProductID,
  3      ProductName,
  4      Price
  5  FROM Product
  6  WHERE Price > (
  7      SELECT AVG(Price)
  8      FROM Product
  9  );

 PRODUCTID PRODUCTNAME                                              PRICE
---------- -------------------------------------------------- ----------
       101 Duke 200                                           200000
       102 Duke 250                                           230000
       103 Duke 390                                           320000

SQL> SELECT
  2      ID,
  3      NAME
  4  FROM Customer
  5  WHERE ID IN (
  6      SELECT Customer_ID
  7      FROM Orders
  8  );

        ID NAME
---------- --------------------
         1 Ramesh
         2 Khilan
         3 Kaushik
         4 Chaitali
         5 Hardik
```

2. Find Products Above Average Price

The inner query calculates the average product price. The outer query displays products whose prices are greater than the average, arranged in descending order.

```
SQL> SELECT
  2      ProductID,
  3      ProductName,
  4      Price
  5  FROM Product
  6  WHERE Price > (
  7      SELECT AVG(Price)
  8      FROM Product
  9  )
 10  ORDER BY Price DESC;

 PRODUCTID PRODUCTNAME                                          PRICE
---------- -------------------------------------------------- ----------
       103 Duke 390                                           320000
       102 Duke 250                                           230000
       101 Duke 200                                           200000

SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      SUM(O.Total_Amount) AS Total_Purchases
  5  FROM Customer C
  6  JOIN Orders O
  7  ON C.ID = O.Customer_ID
  8  GROUP BY C.ID, C.NAME
  9  HAVING SUM(O.Total_Amount) = (
 10      SELECT MAX(Total_Purchase)
 11      FROM (
 12          SELECT SUM(Total_Amount) AS Total_Purchase
 13          FROM Orders
 14          GROUP BY Customer_ID
 15      )
 16  );
```

3. Identify Customers with Maximum Purchases

The nested query calculates each customer's total purchase amount and finds the maximum. The report displays the customer whose total purchase matches the maximum purchase amount.

```
SQL> SELECT
  2      ProductID,
  3      ProductName,
  4      Price
  5  FROM Product
  6  WHERE Price > (
  7      SELECT AVG(Price)
  8      FROM Product
  9  )
 10  ORDER BY Price DESC;

 PRODUCTID PRODUCTNAME                                          PRICE
---------- -------------------------------------------------- ----------
       103 Duke 390                                           320000
       102 Duke 250                                           230000
       101 Duke 200                                           200000

SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      SUM(O.Total_Amount) AS Total_Purchases
  5  FROM Customer C
  6  JOIN Orders O
  7  ON C.ID = O.Customer_ID
  8  GROUP BY C.ID, C.NAME
  9  HAVING SUM(O.Total_Amount) = (
 10      SELECT MAX(Total_Purchase)
 11      FROM (
 12          SELECT SUM(Total_Amount) AS Total_Purchase
 13          FROM Orders
 14          GROUP BY Customer_ID
 15      )
 16  );
```

```
        ID NAME                 TOTAL_PURCHASES
---------- -------------------- ---------------
         5 Hardik                        215000

SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      O.Order_ID,
  5      O.Order_Date,
  6      P.ProductID,
  7      P.ProductName,
  8      OD.Quantity,
  9      OD.Unit_Price,
 10      OD.Subtotal
 11  FROM Customer C
 12  JOIN Orders O
 13  ON C.ID = O.Customer_ID
 14  JOIN Order_Details OD
 15  ON O.Order_ID = OD.Order_ID
 16  JOIN Product P
 17  ON OD.Product_ID = P.ProductID
 18  ORDER BY O.Order_ID;

        ID NAME                 ORDER_ID ORDER_DAT  PRODUCTID
---------- -------------------- ---------- --------- ----------
PRODUCTNAME                                        QUANTITY UNIT_PRICE
-------------------------------------------------- ---------- ----------
  SUBTOTAL
----------
         1 Ramesh                     5001 21-SEP-26        101
Duke 200                                           1     190000
    190000

         2 Khilan                     5002 21-SEP-26        102
Duke 250                                           2       6250
```

4. Complex Business Queries – Order Details

This query combines Customer, Orders, Order_Details, and Product tables to display customer details, order date, product, quantity, unit price, and subtotal.

```
        ID NAME                 TOTAL_PURCHASES
---------- -------------------- ---------------
         5 Hardik                        215000

SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      O.Order_ID,
  5      O.Order_Date,
  6      P.ProductID,
  7      P.ProductName,
  8      OD.Quantity,
  9      OD.Unit_Price,
 10      OD.Subtotal
 11  FROM Customer C
 12  JOIN Orders O
 13  ON C.ID = O.Customer_ID
 14  JOIN Order_Details OD
 15  ON O.Order_ID = OD.Order_ID
 16  JOIN Product P
 17  ON OD.Product_ID = P.ProductID
 18  ORDER BY O.Order_ID;

        ID NAME                 ORDER_ID ORDER_DAT  PRODUCTID
---------- -------------------- ---------- --------- ----------
PRODUCTNAME                                        QUANTITY UNIT_PRICE
-------------------------------------------------- ---------- ----------
  SUBTOTAL
----------
         1 Ramesh                     5001 21-SEP-26        101
Duke 200                                           1     190000
    190000

         2 Khilan                     5002 21-SEP-26        102
Duke 250                                           2       6250
```

```
        ID NAME                 ORDER_ID ORDER_DAT  PRODUCTID
---------- -------------------- ---------- --------- ----------
PRODUCTNAME                                          QUANTITY UNIT_PRICE
-------------------------------------------------- ---------- ----------
  SUBTOTAL
----------
         1 Ramesh                     5001 21-SEP-26        101
Duke 200                                                     1     190000
    190000

         2 Khilan                     5002 21-SEP-26        102
Duke 250                                                     2       6250
     12500

        ID NAME                 ORDER_ID ORDER_DAT  PRODUCTID
---------- -------------------- ---------- --------- ----------
PRODUCTNAME                                          QUANTITY UNIT_PRICE
-------------------------------------------------- ---------- ----------
  SUBTOTAL
----------

         3 Kaushik                    5003 20-SEP-26        103
Duke 390                                                     1      85000
     85000

         4 Chaitali                   5004 20-SEP-26        104
Helmet                                                       1       4200

        ID NAME                 ORDER_ID ORDER_DAT  PRODUCTID
---------- -------------------- ---------- --------- ----------
PRODUCTNAME                                          QUANTITY UNIT_PRICE
-------------------------------------------------- ---------- ----------
  SUBTOTAL
----------
      4200

         5 Hardik                     5005 19-SEP-26        105
Bike Gloves                                                  1     215000
    215000
```

5. Total Quantity Sold by Product

The query uses SUM() and GROUP BY to calculate the total quantity sold for every product, sorted from highest to lowest quantity.

```
SQL> SELECT
  2      P.ProductID,
  3      P.ProductName,
  4      SUM(OD.Quantity) AS Total_Quantity_Sold
  5  FROM Product P
  6  JOIN Order_Details OD
  7  ON P.ProductID = OD.Product_ID
  8  GROUP BY P.ProductID, P.ProductName
  9  ORDER BY Total_Quantity_Sold DESC;

 PRODUCTID PRODUCTNAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       102 Duke 250
                 2

       105 Bike Gloves
                 1

       104 Helmet
                 1
```

```
 PRODUCTID PRODUCTNAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       101 Duke 200
                 1

       103 Duke 390
                 1


SQL> SELECT
  2      P.Category,
  3      SUM(OD.Subtotal) AS Total_Sales
  4  FROM Product P
  5  JOIN Order_Details OD
  6  ON P.ProductID = OD.Product_ID
  7  GROUP BY P.Category
  8  ORDER BY Total_Sales DESC;

CATEGORY                       TOTAL_SALES
------------------------------ -----------
Bike                                287500
Accessories                         219200
```

6. Category-wise Sales Report

This report groups product sales by category and displays total sales for each category in descending order.

```
 PRODUCTID PRODUCTNAME
---------- --------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
       101 Duke 200
                   1

       103 Duke 390
                   1


SQL> SELECT
  2      P.Category,
  3      SUM(OD.Subtotal) AS Total_Sales
  4  FROM Product P
  5  JOIN Order_Details OD
  6  ON P.ProductID = OD.Product_ID
  7  GROUP BY P.Category
  8  ORDER BY Total_Sales DESC;

CATEGORY                    TOTAL_SALES
--------------------------- -----------
Bike                             287500
Accessories                      219200
```

7. Customer Purchase Report

The report combines customer and order data to show total orders, total purchase amount, and average order value for each customer.

```
SQL> SELECT
  2      C.ID,
  3      C.NAME,
  4      COUNT(O.Order_ID) AS Total_Orders,
  5      SUM(O.Total_Amount) AS Total_Purchase_Amount,
  6      AVG(O.Total_Amount) AS Average_Order_Value
  7  FROM Customer C
  8  JOIN Orders O
  9  ON C.ID = O.Customer_ID
 10  GROUP BY C.ID, C.NAME
 11  ORDER BY Total_Purchase_Amount DESC;

        ID NAME                 TOTAL_ORDERS TOTAL_PURCHASE_AMOUNT
---------- -------------------- ------------ --------------------
AVERAGE_ORDER_VALUE
-------------------
         5 Hardik                          1               215000
             215000

         1 Ramesh                          1               190000
             190000

         3 Kaushik                         1                85000
              85000


        ID NAME                 TOTAL_ORDERS TOTAL_PURCHASE_AMOUNT
---------- -------------------- ------------ --------------------
AVERAGE_ORDER_VALUE
-------------------
         2 Khilan                          1                13000
              13000
```

8. Products Whose Sales Are Above Average

The nested query calculates total sales for each product and their average. The outer query displays products whose sales exceed the average product sales.

```
SQL> SELECT
  2      P.ProductID,
  3      P.ProductName,
  4      SUM(OD.Subtotal) AS Total_Sales
  5  FROM Product P
  6  JOIN Order_Details OD
  7  ON P.ProductID = OD.Product_ID
  8  GROUP BY P.ProductID, P.ProductName
  9  HAVING SUM(OD.Subtotal) > (
 10      SELECT AVG(Product_Sales)
 11      FROM (
 12          SELECT SUM(Subtotal) AS Product_Sales
 13          FROM Order_Details
 14          GROUP BY Product_ID
 15      )
 16  )
 17  ORDER BY Total_Sales DESC;

 PRODUCTID PRODUCTNAME                                      TOTAL_SALES
---------- ------------------------------------------------ -----------
       105 Bike Gloves                                           215000
       101 Duke 200                                              190000
```

RESULT

Thus, advanced SQL queries using subqueries and nested queries were implemented successfully in the Duke Bike Sales and Customer Analytics System. Products above average price, customers with maximum purchases, complex business queries, and advanced sales reports were generated.